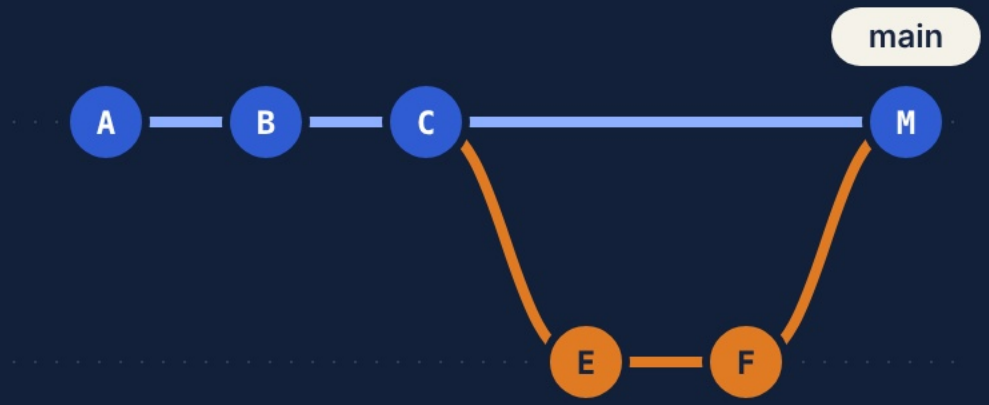
A VISUAL GUIDE · GIT & GITHUB

The GitHub workflow, animated

What happens to your files at every step — with the git command that does it, and what to say to Codex instead.

```
$ git save "Add figure 3"
```

SAY *“save: added figure 3”*



branch deleted; the history stays

Companion to the github-guide folder · verbs: status · save · sync · start · propose · finish · undo

HOW TO READ THIS DECK

Every slide: what moves, the command, the words

WHAT MOVES	WHAT IT MEANS
notes.md modified	**A changed file, not yet committed** Orange bar = work in progress: modified (edited), then staged (picked for the next commit).
B 9f8e7d6 Add data import	**A commit: snapshot + message + parent** Blue dot = recorded history: a letter (our nickname for it), the short hash, the message.
HEAD main new-figures	**Branch names; HEAD = where you are** A branch is a movable name for one commit. main is the official line; new-figures is a branch you start.
	**Between slides the cards glide: that motion is what the command does** Watch the stage first, then read the caption below it.

The git command. Type it in Terminal, or watch Codex run it.

```
$ git push
```

The words to say to Codex instead. With the git-workflow skill, the verb alone works: save, sync, propose…

SAY *“Push my commits”*

Guide: README.md · Part 1 · Part 2 §3

THE FOUR PLACES

Four places a change lives

ON YOUR MAC

ON GITHUB.COM



A change travels left to right: edit, then **add**, **commit**, **push**. **pull** brings GitHub's new commits back to your Mac.

```
$ git add · git commit · git push
```

SAY *"save"* adds, commits, pulls and pushes

Guide: Part 1 §3–4

SAVING WORK · STEP 1 OF 4

Edit: your files change, nothing is recorded

ON YOUR MAC

ON GITHUB.COM

Working directory
your files, as edited

- `model.R` modified
- `notes.md` modified

git status lists both as "Changes not staged for commit"

Staging area
the next commit

empty

Local repository
your history

- B `9f8e7d6` Add data import
- A `1a2b3c4` Initial commit

GitHub
shared copy: origin

- B `9f8e7d6` Add data import
- A `1a2b3c4` Initial commit

You **edit** model.R and notes.md in RStudio. Git notices the changes but records nothing until you commit.

```
$ git status
```

SAY *"status"*

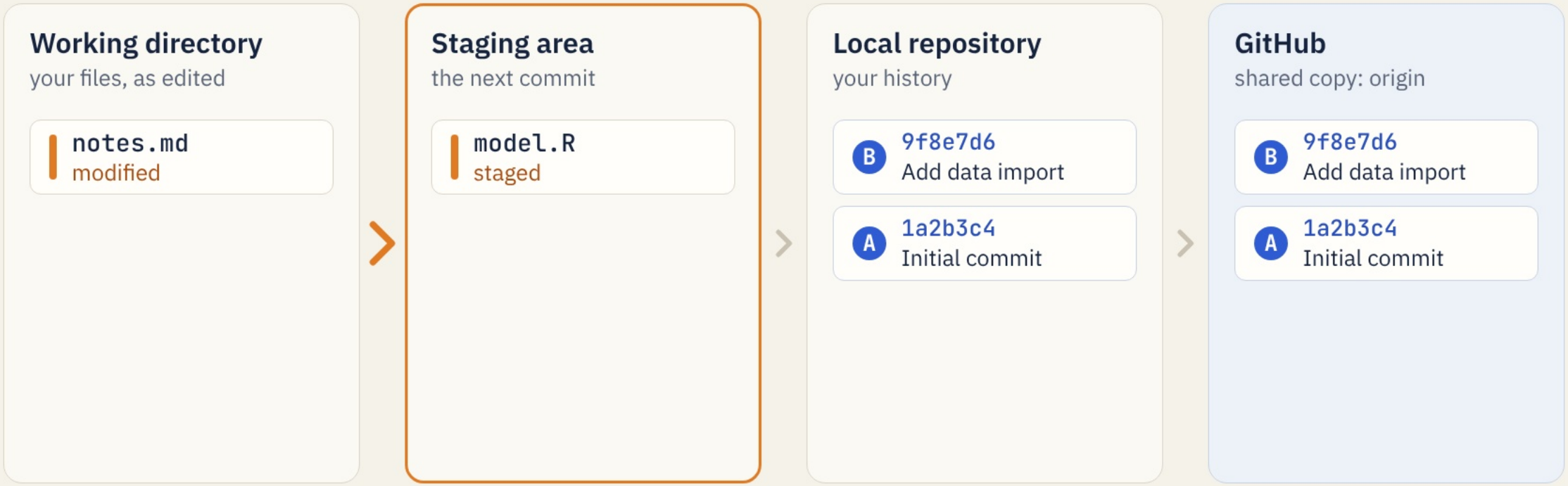
Guide: Part 1 §3 · Part 2 §4.3

SAVING WORK · STEP 2 OF 4

Stage: pick what goes into the next commit

ON YOUR MAC

ON GITHUB.COM



git add moves the change to model.R into the staging area. notes.md stays out: it will not be in this commit.

```
$ git add model.R
```

SAY *“Stage model.R only, not notes.md”*

Guide: Part 1 §3 (the staging area) · Part 2 §4.4

SAVING WORK · STEP 3 OF 4

Commit: a snapshot enters your history

ON YOUR MAC

Working directory
your files, as edited

`notes.md`
modified

Never staged,
so not in C.

Staging area
the next commit

empty

Local repository
your history

C `c3f9a1e` Fix sign error

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

ON GITHUB.COM

GitHub
shared copy: origin

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

No C here yet. git status:
“ahead of 'origin/main'
by 1 commit”

The staged change becomes commit **C**: a snapshot with a message, an author, a time and a pointer to B. It exists only on your Mac.

```
$ git commit -m "Fix sign error"
```

SAY *“Commit it: fix sign error”*

Guide: Part 1 §2–3 · Part 2 §4.4

SAVING WORK · STEP 4 OF 4

Push: upload your new commits to GitHub

ON YOUR MAC

ON GITHUB.COM

Working directory
your files, as edited

`notes.md`
modified

Not committed,
so not pushed.

Staging area
the next commit

empty

Local repository
your history

C `c3f9a1e` Fix sign error

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

GitHub
shared copy: origin

C `c3f9a1e` Fix sign error

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

git status: “up to date with 'origin/main'”

git push copies commit C to GitHub. Both histories now read A, B, C. notes.md is still only on your Mac, uncommitted.

```
$ git push
```

SAY *“Push my commits”*

Guide: Part 1 §4 · Part 2 §4.4

SAVING WORK · THE SHORTCUT

save = add + commit + pull + push

`$ git add -A` + `$ git commit -m "…"` + `$ git pull` + `$ git push`

stage all changes (except ignored files)

record the snapshot with your message

merge in GitHub's new commits

upload your commits to GitHub

= `$ git save "Fix sign error"` *or* `SAY “save: fixed the sign error”`

in Terminal, with the alias from helpers/

to Codex, with the git-workflow skill

Commit first, then pull: GitHub's new commits merge in before the push. With nothing new to commit, save just pulls and pushes.

Guide: Part 2 §4.4 · helpers/gitconfig-aliases

SYNCING · STEP 1 OF 2

Someone pushed: GitHub is one commit ahead

ON YOUR MAC

Working directory
your files, as edited

`notes.md`
modified

Staging area
the next commit

empty

Local repository
your history

C `c3f9a1e` Fix sign error

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

status, after fetch:
behind by 1 commit

ON GITHUB.COM

GitHub
shared copy: origin

D `5e6f7a8` Edit README

C `c3f9a1e` Fix sign error

B `9f8e7d6` Add data import

A `1a2b3c4` Initial commit

Codex (cloud) or your other laptop pushed commit **D**. Your Mac doesn't know until it fetches: then status says "behind by 1".

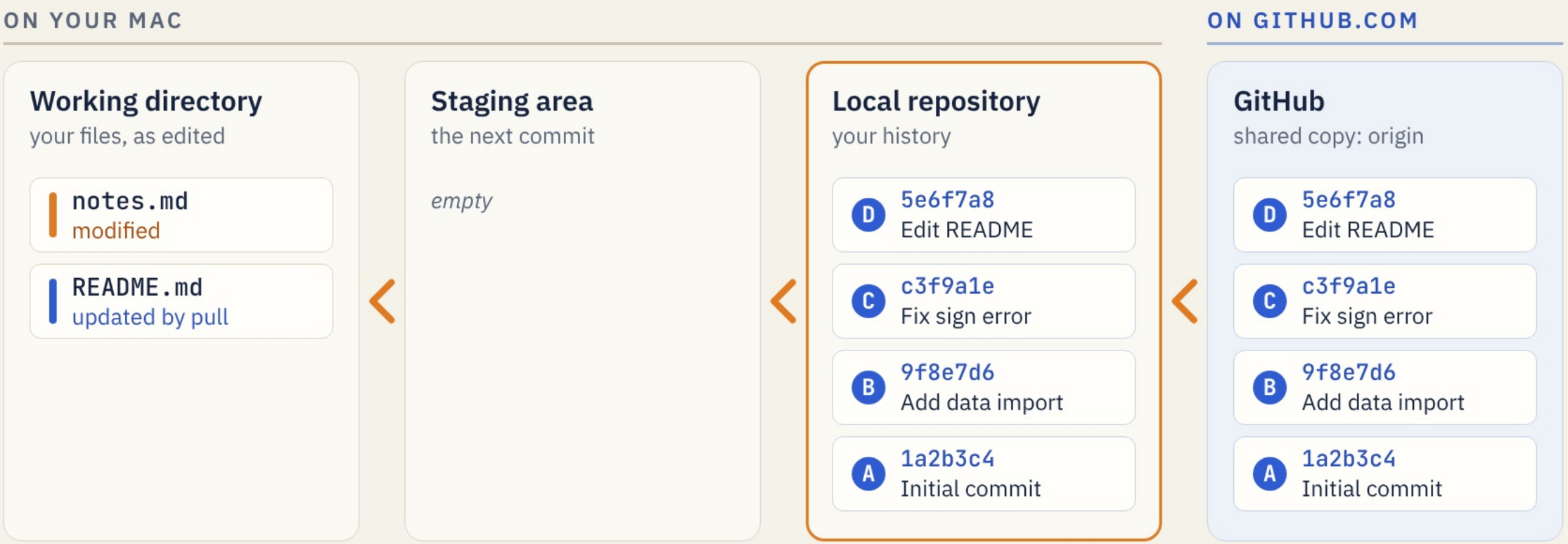
```
$ git fetch && git status
```

SAY *"status"* fetches first, then reports ahead / behind

Guide: Part 1 §4 · Part 2 §4.3

SYNCING · STEP 2 OF 2

Pull: bring GitHub's commits to your Mac



git pull = fetch + merge: commit D joins your history and README.md updates on disk. Your uncommitted notes.md is untouched.

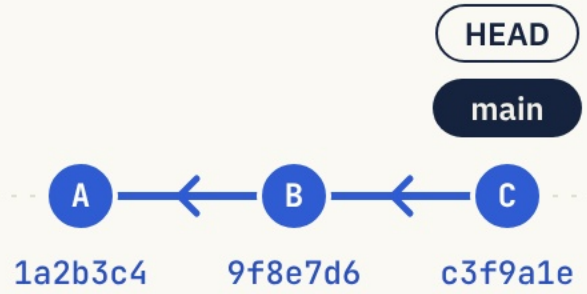
```
$ git pull
```

SAY *"sync"*

Guide: Part 1 §4 · Part 2 §4.5 · Terminal shortcut: git sync

main: the official line of history

HISTORY OF YOUR REPOSITORY



Each commit points to its parent: A ← B ← C.
A branch is a name pointing at one commit, like a bookmark in a ledger.

main is a name that points at the newest commit, C. HEAD says you are on main. Start each piece of work from an up-to-date main.

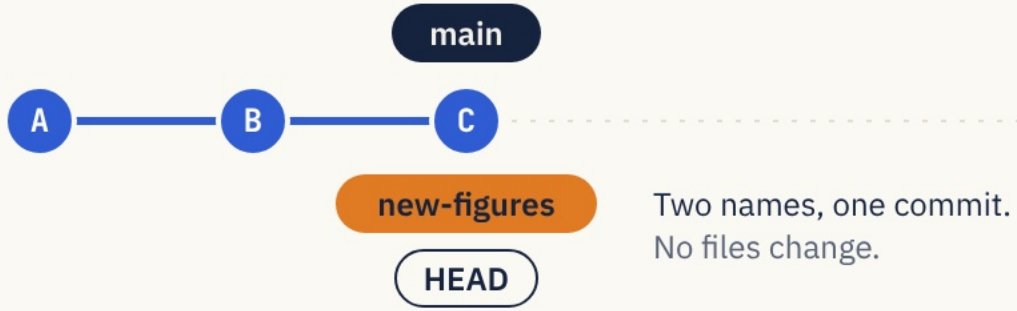
```
$ git switch main && git pull
```

SAY *“sync”*

Guide: Part 1 §2 · Part 2 §4.5

Start: a new branch name on the same commit

HISTORY OF YOUR REPOSITORY



Two names, one commit.
No files change.

git switch -c creates the name new-figures on C and moves HEAD to it. No commit is made, no file changes.

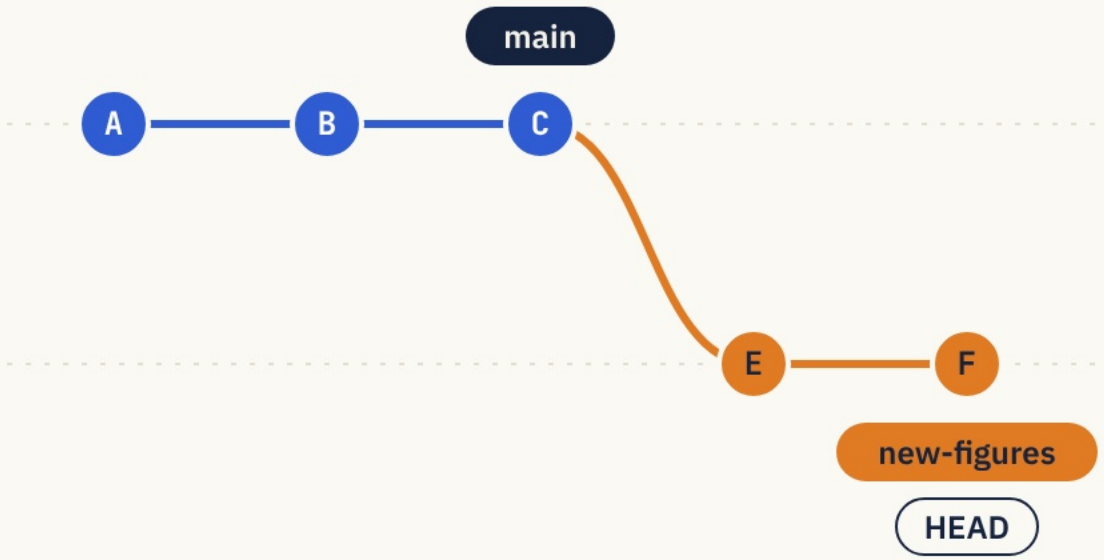
```
$ git switch -c new-figures
```

SAY *“start new-figures”*

Guide: Part 2 §4.6 · Terminal shortcut: git start new-figures

Work on the branch: only its name moves

HISTORY OF YOUR REPOSITORY



HEAD = the branch you are on. New commits go where HEAD is.

Each commit on the branch moves the **new-figures** name forward. **main** still points at C, untouched.

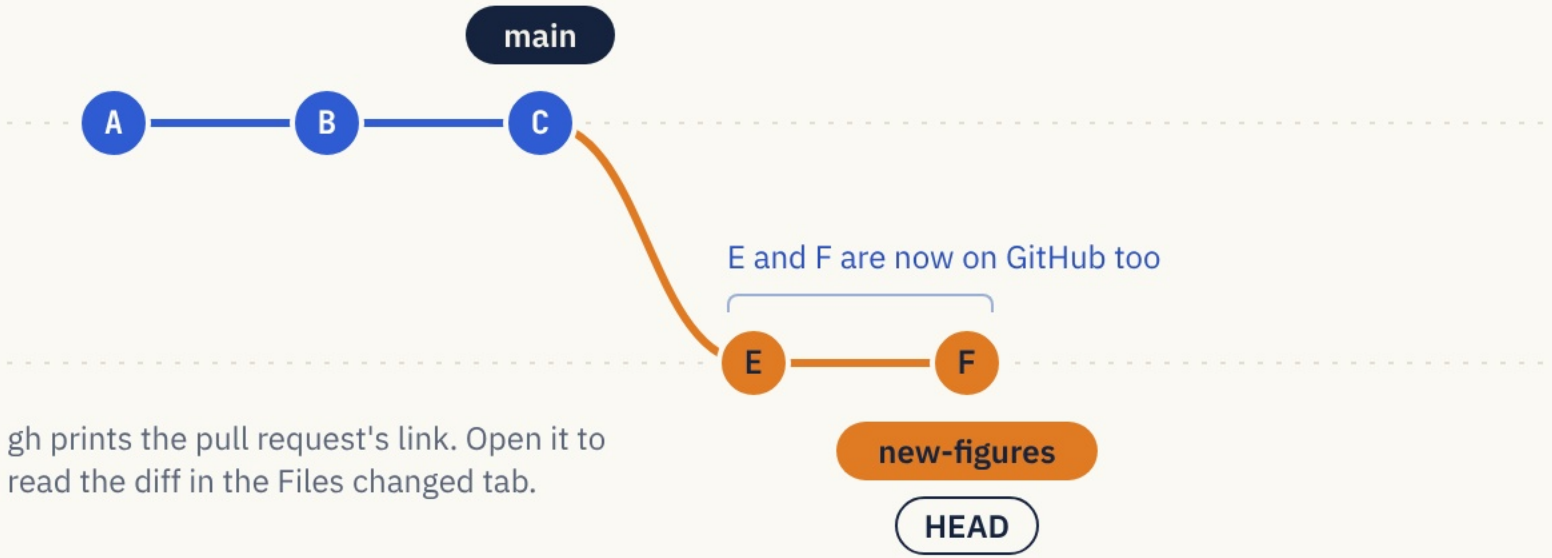
```
$ git add -A && git commit -m "Add fig 3"
```

SAY *“Commit this as ‘Add fig 3’”*

Guide: Part 1 §2 (branch, HEAD) · Part 2 §4.6

Propose: push the branch, open a pull request

HISTORY OF YOUR REPOSITORY



PULL REQUEST #12

Add new figures

new-figures → main

2 commits · +84 −12

E F

Open

Waiting for review

git push uploads E and F; **gh pr create** opens a pull request: a GitHub page proposing to merge new-figures into main, with the diff.

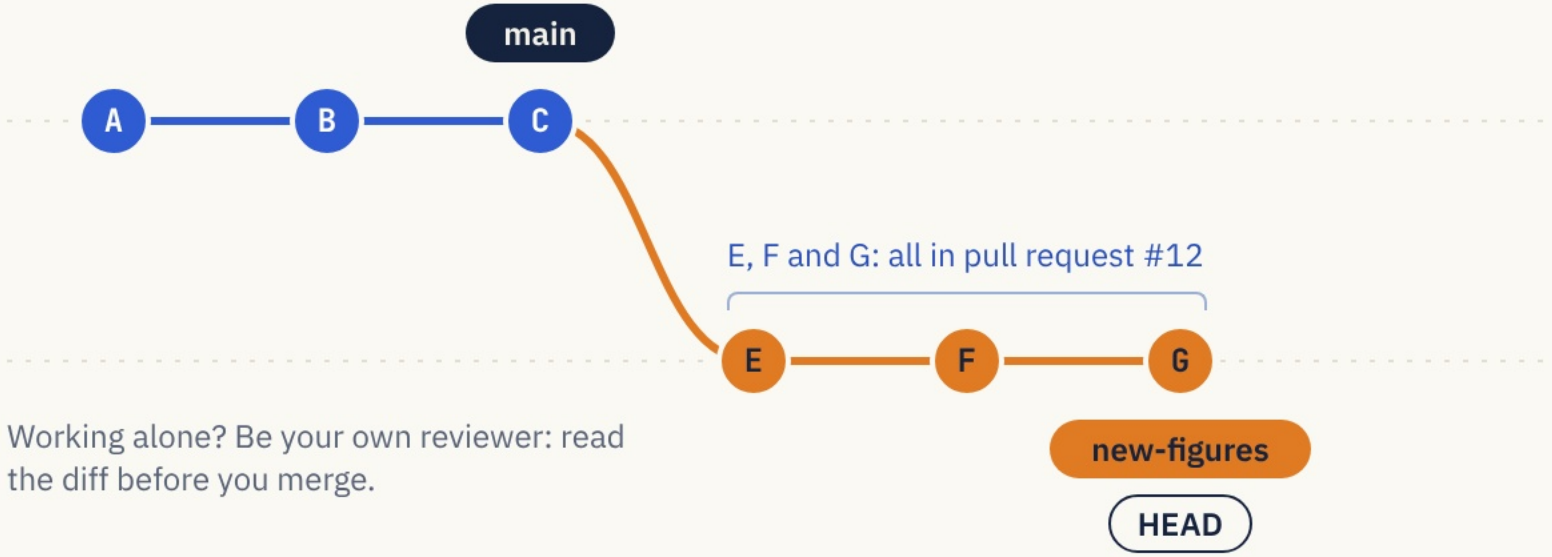
```
$ git push && gh pr create --fill
```

SAY *“propose”*

Guide: Part 1 §6 · Part 2 §4.6 · Terminal shortcut: git propose

Review: each push updates the same pull request

HISTORY OF YOUR REPOSITORY



PULL REQUEST #12

Add new figures

`new-figures` → `main`

3 commits · +97 −15

E F G

Open

Reviewer: “Label the y-axis.”
→ **fixed in G**

A reviewer asks for a change. You commit **G** on the same branch and push: the pull request picks it up automatically.

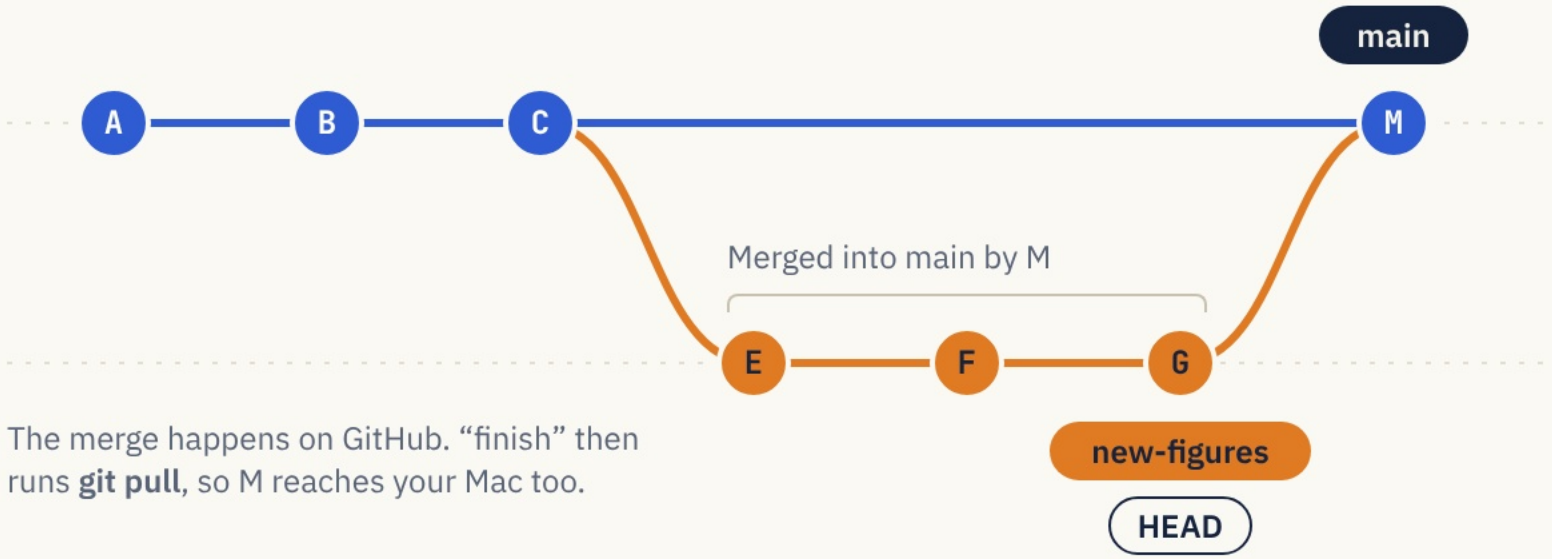
```
$ git save "Label y-axis"
```

SAY *“save: labelled the y-axis”*

Guide: Part 2 §4.6

Finish: merge the pull request into main

HISTORY OF YOUR REPOSITORY



The merge happens on GitHub. “finish” then runs **git pull**, so M reaches your Mac too.

PULL REQUEST #12

Add new figures

new-figures → main

3 commits · +97 −15

E F G

Merged

3 commits merged into main

Merging makes commit **M** with two parents, C and G. main moves to M: the figures are now in the official version.

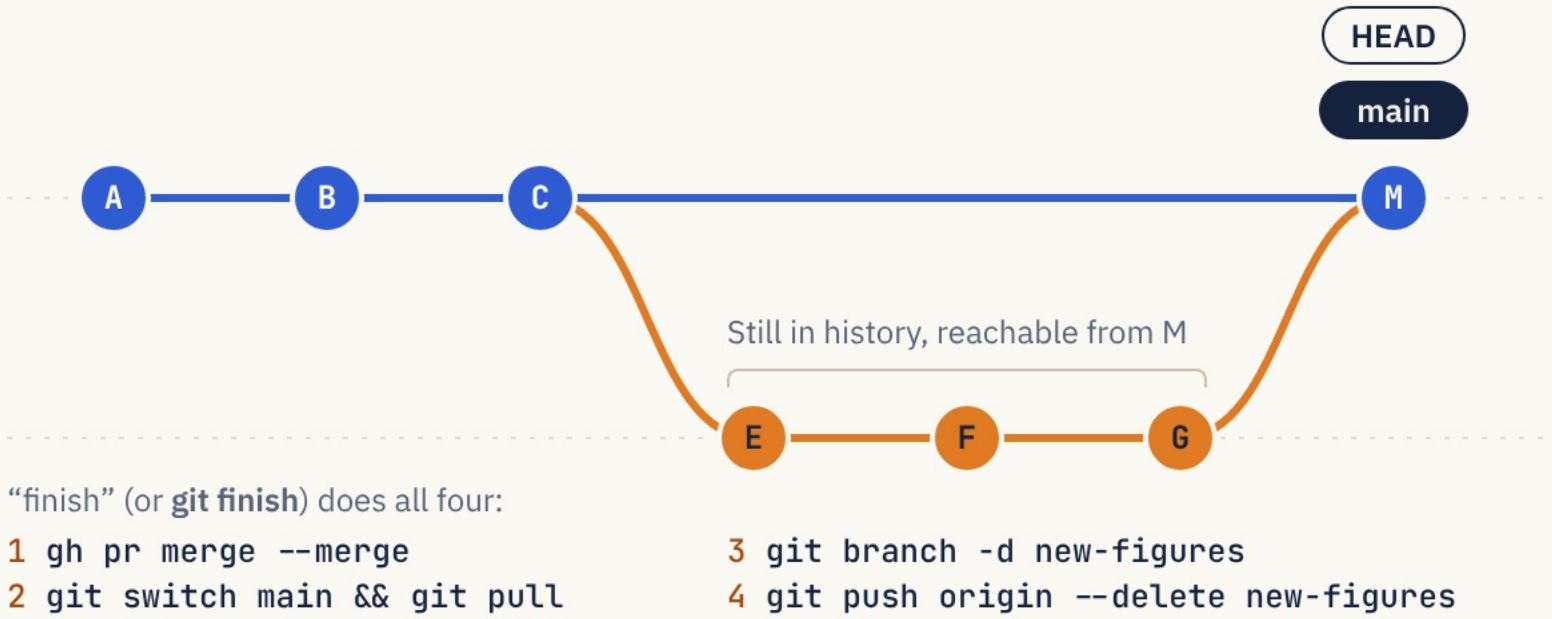
```
$ gh pr merge --merge
```

SAY *“finish”*

Guide: Part 1 §5 · Part 2 §4.7

Clean up: delete the branch name, keep its commits

HISTORY OF YOUR REPOSITORY



PULL REQUEST #12

Add new figures

new-figures → main

3 commits · +97 −15

E F G

Merged

Branch deleted

Back on main and up to date, you delete the name new-figures. Its commits E, F, G stay in history, reachable through M.

```
$ git branch -d new-figures
```

SAY *"finish"* does steps 6 and 7 together

Guide: Part 2 §4.7 · Terminal shortcut for steps 6–7: git finish

CONFLICTS · STEP 1 OF 2

Conflict: both sides changed the same line

LINE 12 OF MODEL.R, TWO VERSIONS

Your commit (on your Mac) — C HEAD

```
11  beta <- coef(fit)
12  alpha <- 0.05
```

Commit D on GitHub — D 5e6f7a8

```
11  beta <- coef(fit)
12  alpha <- 0.10
```

yours

GitHub's

model.R after git pull

```
CONFLICT (content): Merge conflict in model.R
12  <<<<<<< HEAD
13  alpha <- 0.05
14  =======
15  alpha <- 0.10
16  >>>>>>> 5e6f7a8
```

Other lines merged on their own: only line 12 needs you.

git pull can't choose between 0.05 and 0.10, so it writes both versions into model.R between markers and stops. Nothing is lost.

```
$ git pull
```

SAY *"sync"*

Guide: Part 1 §5 · Part 2 §4.9

CONFLICTS · STEP 2 OF 2

Resolve: keep one version, delete the markers

1 · YOU EDIT THE FILE

model.R edited by you

```
11  beta <- coef(fit)
12  alpha <- 0.10
```

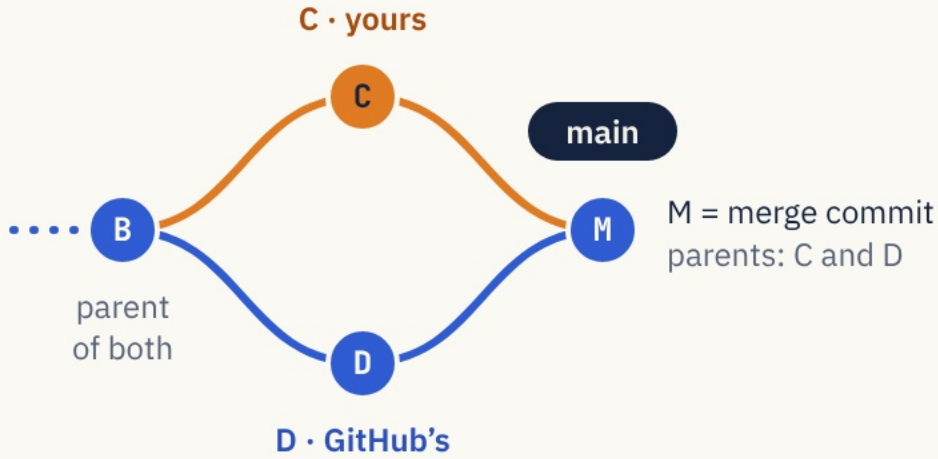
No markers left: ready to stage and commit.

DELETED BY YOU

- ~~<<<<<<< HEAD~~
- ~~alpha <- 0.05~~
- ~~=======~~
- ~~>>>>>>> 5e6f7a8~~

Edit the block to what you want (here 0.10) and delete the three marker lines. Stage and commit: that concludes the merge. Then push, or just run **git save**.

2 · THE COMMIT CONCLUDES THE MERGE




```
$ git add model.R && git commit --no-edit
```

SAY *"Resolve the conflict: keep 0.10"*

Guide: Part 2 §4.9 · git save refuses while markers remain

UNDO · FOUR TOOLS

Undo: pick the tool by where the change is

ON YOUR MAC

Working directory
your files, as edited

Throw away edits to a file

```
git restore model.R
```

The file goes back to its last commit.

Warning: edits are lost.

Staging area
the next commit

Unstage a file

```
git restore --staged notes.md
```

The file just leaves the next commit.

Safe: your edits stay.

Local repository
your history

Undo the last commit

```
git uncommit
```

Changes stay staged.
Same as:
`reset --soft HEAD~1`

Safe: refuses if pushed.

ON GITHUB.COM

GitHub
shared copy: origin

Undo a pushed commit

```
git revert <hash>
```

Adds an inverse commit, which you push.

Safe: history is kept.

Gentlest first. Once a commit is on GitHub, only **revert** is safe: it adds a new commit instead of rewriting history.

```
$ git revert --no-edit <hash> && git push
```

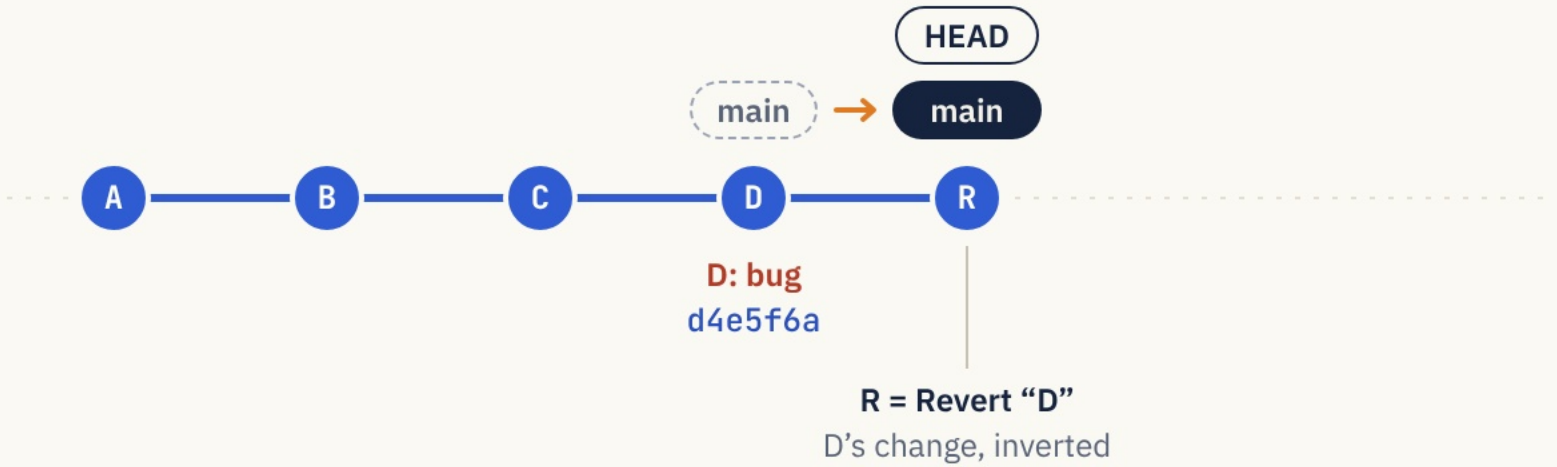
SAY *“Undo commit 5e6f7a8, it’s pushed”*

Guide: Part 1 §7 · Part 2 §4.10

UNDO · REVERT

Revert: undo a pushed commit with its inverse

HISTORY OF MAIN, ALREADY ON GITHUB



THE TWO ENTRIES

D + R = no change

D d4e5f6a bug

```
- lookback <- 250
+ lookback <- 25
```

R new commit

```
- lookback <- 25
+ lookback <- 250
```

Like a reversing journal entry: the audit trail survives.

git revert adds a new commit R that undoes D’s changes. History keeps both D and R, so collaborators and GitHub stay consistent.

```
$ git revert --no-edit d4e5f6a && git push
```

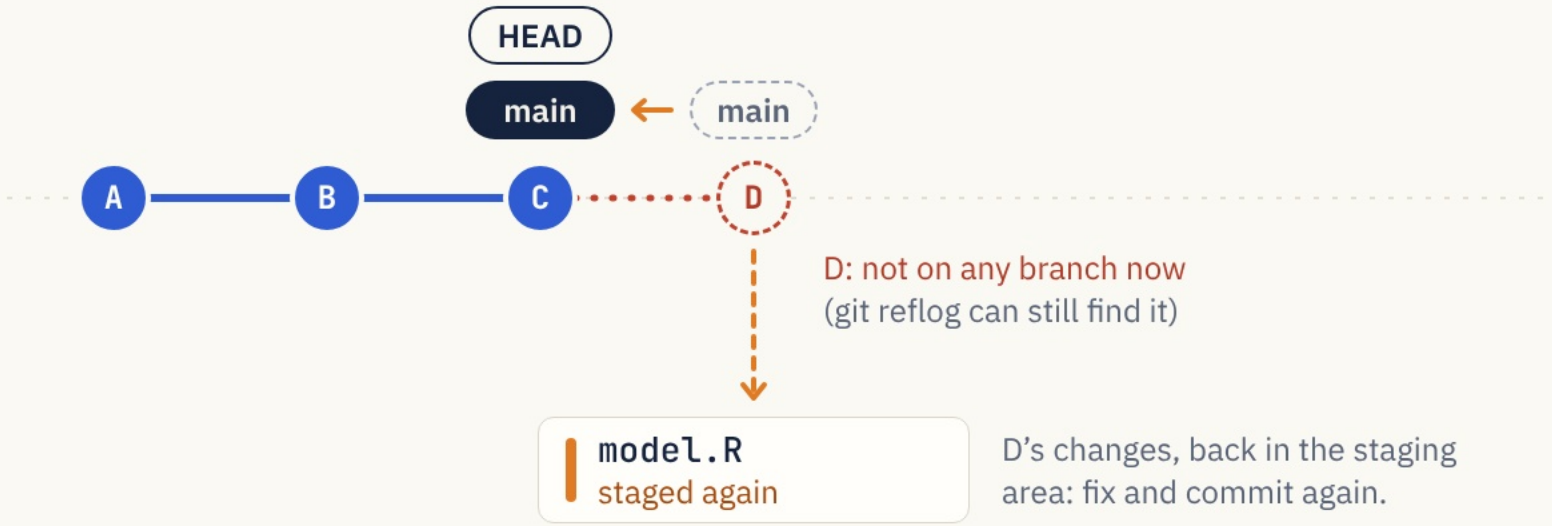
SAY *“Undo commit D; it’s already pushed”*

Guide: Part 1 §7 · Part 2 §4.10

UNDO · RESET

Reset: move the branch back — only before pushing

HISTORY OF MAIN, NOT YET PUSHED



WARNING

After a push, use revert.

Once D is on GitHub, others may have it: only revert is safe.

`reset --soft`
keeps D's changes, staged.

`reset --hard`
also deletes uncommitted edits. They are lost.

reset --soft HEAD~1 moves main back one commit; D's changes return to the staging area. HEAD~1 = the commit before the current one.

```
$ git reset --soft HEAD~1
```

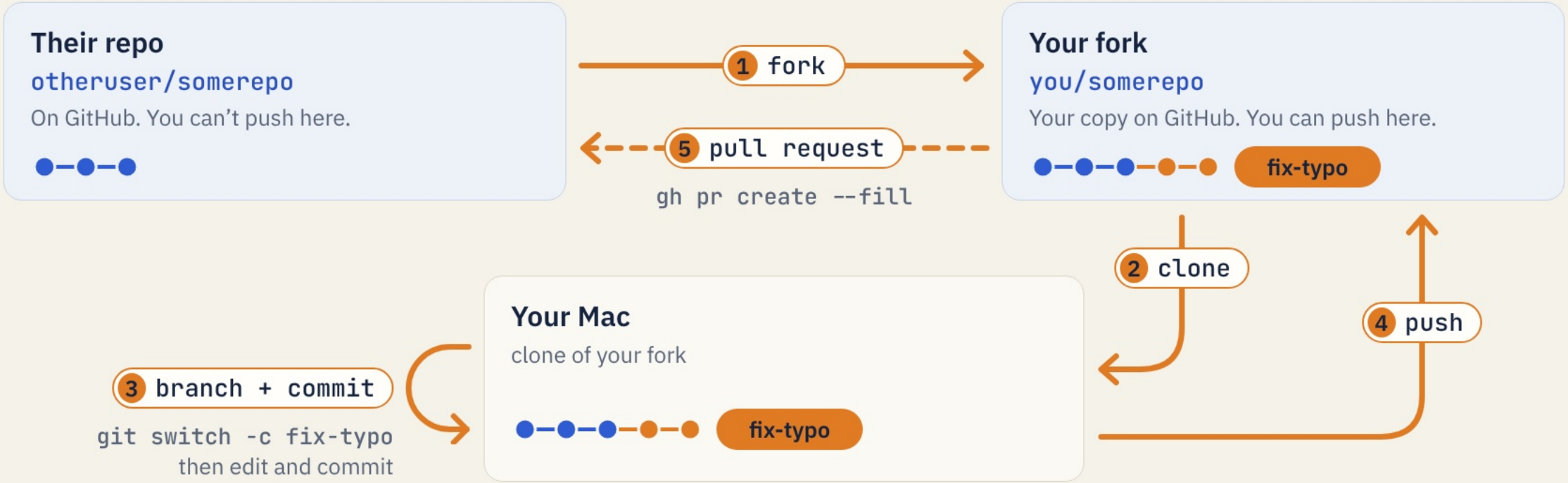
SAY *"Undo my last commit, I haven't pushed"*

Guide: Part 1 §7 · Terminal shortcut: git uncommit (refuses if pushed)

CONTRIBUTING · FORKS

Contribute to someone else’s repo: fork, then propose

THREE COPIES OF ONE REPOSITORY



A fork is your own copy on GitHub. You push to it, then open a pull request back to the original. Catch up later with **gh repo sync**.

```
$ gh repo fork otheruser/somerepo --clone
```

SAY *“contribute otheruser/somerepo”*

Guide: Part 1 §6 · Part 2 §4.8

RECAP

Cheat sheet: the verbs

VERB	SAY TO CODEX	TERMINAL	WHAT RUNS
status	*“status”*	`git st`	git status, git log
save	*“save: <what changed>”*	`git save "msg"`	add, commit, pull, push
sync	*“sync”*	`git sync`	git pull
start	*“start new-figures”*	`git start <topic>`	switch main, pull, switch -c
propose	*“propose”*	`git propose`	push, gh pr create
finish	*“finish”*	`git finish`	gh pr merge, switch main, pull, delete branch
undo	*“undo ...” (say what)*	`git uncommit`	restore · reset --soft · revert
new-repo	*“new-repo sizing-paper”*	—	git init, commit, gh repo create
contribute	*“contribute owner/repo”*	—	gh repo fork --clone, switch -c

Say works in Codex with the git-workflow skill installed: the verb alone is enough.
Terminal shortcuts are the aliases in helpers/gitconfig-aliases. helpers/install.sh sets up both.

Guide: Part 2 §4 · helpers/README.md

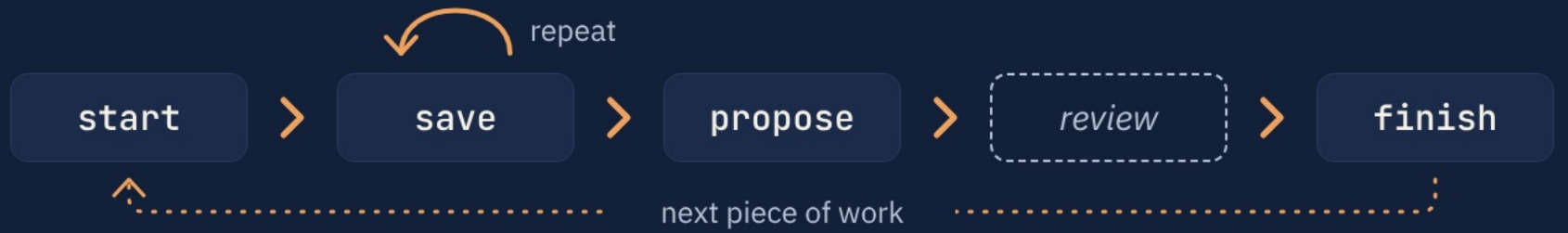
THE WHOLE WORKFLOW

Three loops cover almost everything

Solo:
your repo, on main



Collaborating:
branch + pull request



Oops:
at any step

`undo` restore · uncommit · revert: pick the tool by where the change is

Pull before you start, push when you stop.

say it · do it yourself

Written guide: github-guide/ in contravariant/presentations (branch github-guide)